Инструкция по развёртыванию и администрированию

Версия 1.0 · 23.09.2026

1. Требования

Параметр	Старт (до 50 пользователей)	Рост (до 500 пользователей)
Сервер приложений	4 vCPU, 8 ГБ RAM, 60 ГБ SSD	2–3 сервера по 4 vCPU / 8 ГБ за балансировщиком
База данных	на том же сервере	отдельный сервер 4 vCPU / 16 ГБ, SSD, реплика для отчётов
ОС	Linux (Ubuntu 22.04+, Debian 12, Astra Linux) с Docker 24+ и Compose v2	
Сеть	домен, TLS-сертификат, исходящий доступ к SMTP/IMAP, api.telegram.org, АТС	

2. Установка

```
git clone <репозиторий> /opt/crm && cd /opt/crm
cp .env.example .env
```

Заполните `.env`:

Переменная	Описание
`SECRET_KEY`	Случайная строка 48+ символов: `python3 -c "import secrets; print(secrets.token_urlsafe(48))"`
`FIELD_ENCRYPTION_KEY`	Ключ шифрования телефонов: `python3 -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"`. **Храните резервную копию ключа отдельно от бэкапов БД — без него зашифрованные поля не восстановить.**
`PUBLIC_BASE_URL`	Внешний адрес, например `https://crm.company.ru` (для ссылок в письмах, трекинга, КП)
`CORS_ORIGINS`	Тот же адрес
`POSTGRES_PASSWORD`, `S3_SECRET_KEY`	Надёжные пароли
`SMTP_*`, `MAIL_FROM`	Почтовый сервер для отправки писем
`IMAP_*`	Ящик для приёма заявок (например, sales@company.ru), `IMAP_ENABLED=true`
`TELEGRAM_BOT_TOKEN`	Токен бота от @BotFather
`TELEPHONY_*`	Параметры АТС, `TELEPHONY_WEBHOOK_SECRET` — секрет вебхука

Переменная	Описание
`STORAGE_BACKEND`, `S3_*`	Для Yandex Object Storage: `S3_ENDPOINT_URL=https://storage.yandexcloud.net`, ключи сервисного аккаунта

Запуск:

```
docker compose up -d --build
docker compose exec backend python -m app.seed      # справочники и администратор admin@crm.local / admin
```

При первом входе администратор обязан сменить пароль. Дальше: «Администрирование → Компания и настройки» (реквизиты, логотип), «Пользователи» (сотрудники получат доступ на email).

Демо-стенд с тестовыми данными: `python -m app.seed --demo` (на пустой базе).

3. HTTPS

Порт `HTTP_PORT` (по умолчанию 8080) закрывается обратным прокси с TLS 1.2+. Пример для nginx на хосте:

```
server {
    listen 443 ssl http2;
    server_name crm.company.ru;
    ssl_certificate     /etc/letsencrypt/live/crm.company.ru/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/crm.company.ru/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    add_header Strict-Transport-Security "max-age=31536000" always;
    client_max_body_size 30m;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
server { listen 80; server_name crm.company.ru; return 301 https://$host$request_uri; }
```

В `docker-compose.yml` для продуктива уберите публикацию портов `8000`, `8025`, `9001`, оставьте только фронтенд на `127.0.0.1:8080`. Mailpit (тестовая почта) в продуктиве не нужен — укажите реальный SMTP.

4. Резервное копирование и восстановление

- Сервис `backup` ежедневно в `BACKUP_HOUR` (UTC) делает `pg_dump` в `./backups`, архивы старше `BACKUP_KEEP_DAYS` (30) удаляются. Первый архив создаётся сразу после запуска.
- Каталог `./backups` и бакет S3 копируйте на внешнее хранилище (rsync / `rclone` / снапшоты диска).
- Ручной бэкап: `./scripts/backup.sh`.

- Восстановление: `./scripts/restore.sh backups/crm_YYYYMMDD_HHMMSS.dump`. Скрипт останавливает приложение, пересоздаёт БД, восстанавливает и запускает приложение, выводит длительность. Для БД в несколько ГБ — минуты, что укладывается в RTO < 1 часа (ТЗ п. 4.2) и критерий < 30 минут (п. 8.2).
- Раз в квартал проверяйте восстановление на тестовом стенде.

5. Обновление версии

```
git pull
docker compose up -d --build     # миграции БД применяются автоматически при старте backend
```

Перед обновлением сделайте `./scripts/backup.sh`. Откат: предыдущий тег образа + восстановление архива.

6. Мониторинг

Что	Как
Доступность	`GET /api/health` → `{"api":"ok","database":"ok","redis":"ok"}`, код 503 при недоступной БД. Подключите к Zabbix / Uptime Kuma / Prometheus blackbox
Логи	`docker compose logs -f backend worker beat`; в продуктиве — драйвер логов `json-file` с ротацией или Loki
Фоновые задачи	Логи `worker`, `beat`: письма, напоминания, IMAP, отчёты
Действия пользователей	«Администрирование → Журнал аудита»
Контейнеры	Healthcheck backend встроен в образ; `restart: unless-stopped` для всех сервисов

7. Масштабирование

```
docker compose up -d --scale backend=3 --scale worker=2
```

При нескольких репликах backend поставьте балансировщик (nginx upstream / облачный LB) и выставьте одинаковый `.env` на всех узлах. Beat запускайте **в одном экземпляре**. Для Kubernetes: Deployment для backend и worker, single-replica Deployment для beat, управляемый PostgreSQL и Redis, S3-хранилище.

8. Интеграции

- **Сайт**: «Администрирование → Интеграции» → создать API-ключ → вставить готовый код формы на сайт. Заявки отправляются на `POST /api/v1/public/leads`.
- **Телефония**: АТС отправляет события (`incoming`, `completed`, `missed`) на `POST /api/v1/integrations/telephony/events` с заголовком `X-Webhook-Secret`. Формат — в Swagger (`TelephonyEvent`). Для click-to-call укажите `TELEPHONY_PROVIDER=generic_http` и URL адаптера АТС, принимающего `{"from": "<внутр. номер>", "to": "<телефон>"}`. Внутренние номера сотрудников задаются в карточке пользователя.
- **1С**: «Счета и оплаты → Выгрузка в 1С» — XML со счетами, актами и контрагентами.

- **Google Calendar / Outlook**: каждый сотрудник берёт персональную ссылку в «Календарь → Синхронизация».

9. Безопасность эксплуатации

- Регулярно обновляйте образы (`docker compose pull && docker compose up -d --build`).
- Ограничьте доступ к серверу по SSH-ключам, закройте все порты, кроме 443.
- Включите 2FA для администраторов и руководителей.
- Ключи `SECRET_KEY` и `FIELD_ENCRYPTION_KEY` храните в менеджере секретов; смена `SECRET_KEY` разлогинит всех пользователей.
- Проводите пентест по OWASP Top 10 перед вводом в эксплуатацию (ТЗ п. 8.2) и после крупных изменений.

10. Нагрузочное тестирование (ТЗ п. 8.2)

```
docker compose run --rm backend sh -c "pip install --user -q locust && \
  python -m locust -f tests/load/locustfile.py --host http://frontend --users 50 --spawn-rate 5 \
  --run-time 5m --headless --csv /tmp/load"
```

На время теста увеличьте `RATE_LIMIT_PER_MINUTE`: шесть демо-учёток при 50 виртуальных пользователях упираются в лимит. Критерий приёмки: 95-й перцентиль времени ответа < 2000 мс.